

FastMCP Documentation Questions

1. Choosing Between `stdio` and `HTTP` Transport

FastMCP supports different transport methods for communicating between an MCP server and its client. I would choose **`stdio`** when the MCP server is running locally and is mainly being used by a local MCP client. For example, if I have my movie MCP server running on my own laptop and I want to use it with Claude Desktop, `stdio` makes sense because the client can start the server as a local process and communicate with it through standard input and output.

FastMCP uses `stdio` by default, so the server can simply be started with:

```
mcp.run()
```

If I wanted to make my movie MCP server available over a network, I would use **`HTTP`** instead. `HTTP` is useful when the server needs to be accessed remotely, when multiple clients need to connect to the same server, or when I want to deploy the server as a network service.

In that case, I would change the `mcp.run()` call to specify the `HTTP` transport and networking options:

```
mcp.run(transport="http", host="127.0.0.1", port=8000)
```

The main change is therefore the `transport` argument. With no argument, FastMCP uses `stdio` by default. With `transport="http"`, FastMCP starts an `HTTP` server that clients can connect to through a URL.

So, for my local movie MCP server I would use `stdio`, while I would switch to `HTTP` if I wanted to deploy the server and allow clients to access it over a network.

2. Plain Text vs Structured Output in FastMCP

FastMCP can return tool results as normal content as well as structured data. The difference mainly depends on what the tool returns and whether FastMCP can represent that result as structured output.

If a tool returns a plain string, FastMCP converts it into a text content block. For example, if my movie tool returns:

```
return "Ryan Gosling plays Ken in Barbie."
```

the result is treated as text. This is useful when the client mainly needs a human-readable response.

On the other hand, if my tool returns an object-like value such as a dictionary or a Pydantic model, FastMCP automatically creates structured content from it. For example:

```
return {
    "title": "Barbie",
    "year": 2023,
    "actor": "Ryan Gosling"
}
```

This allows the client to receive the individual fields as machine-readable data instead of only receiving one block of text.

This distinction matters because structured output is much easier for a client to process programmatically. For example, if my movie MCP tool returns information about a movie as a dictionary, the client can directly access fields such as the title, year, and actor instead of having to extract those values from a text response.

FastMCP also supports structured output for primitive values such as integers when an appropriate return type annotation or output schema is provided. Without such a schema, a simple return value such as an integer is treated as traditional content.

Therefore, I would use a plain string when I mainly want to return a readable message, and I would use a dictionary or Pydantic model when I want the client to work with the returned data in a structured way.